Tech McRAE

An HR information system built to manage employee attendance and leave requests. It features role-based access with dedicated portals for employees and administrators. Employees can clock in and out with office or remote modes, track daily hours, and submit leave requests. Administrators can monitor attendance analytics, review leave pipelines with Gantt timelines, and manage employee records.

tech stack

- Laravel Framework (HTML, CSS, JS)
- Docker

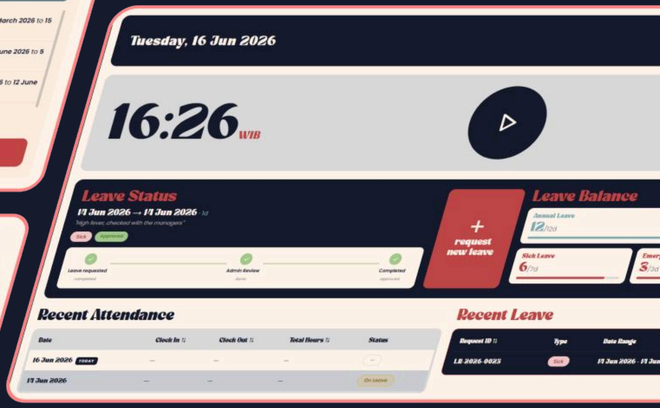
links

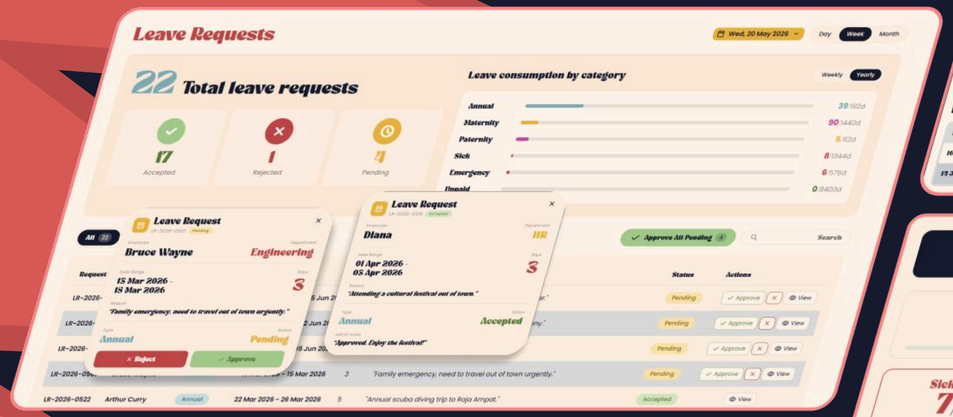
chrysan95

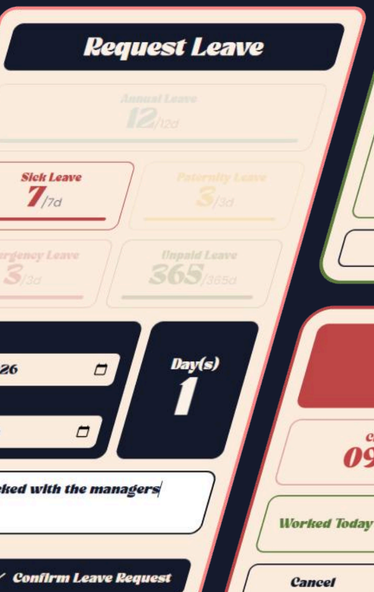




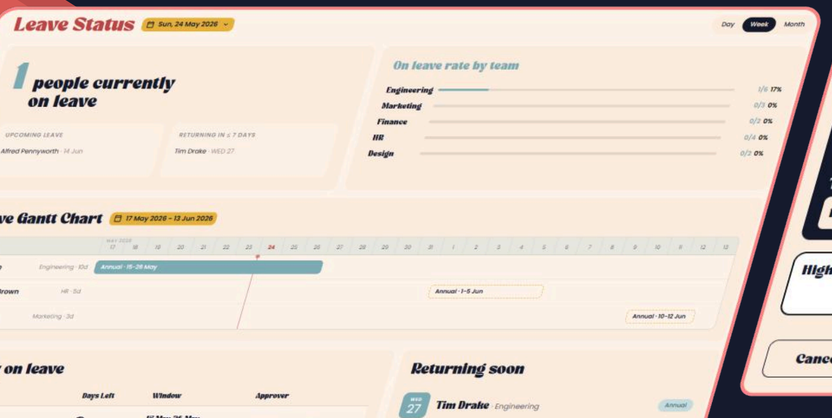














goals

- Build a human resource information system within the assignment’s deadline
- Build a human resource information system with easy to interpret dashboard
- Build a human resource information system that includes gamified design and colorful colors to differ from other human resource information systems
- Learn docker using an already made project

difficulties

- Human resource information system typically has a lot of features other than attendances and leave requests
- CSRF didn’t work on local deployment using XAMPP despite it being very close to the deadline
  Fix: For demonstration purposes, CSRF tokens were not validated
- Wide screen was having problems as the UI were not that responsive
- User were able to request a leave even with overlapping dates
- User were able to request a leave when they have less quota than the days they were requesting

what i learned

- I learned how to prioritize features based on the time that was given and add features along the way
- I learned how to gamify an information system which typically included very gray and uninteresting UI
- I learned how to use charts.js to make data visualizations on the dashboard
- I learned to test my own application
- I learned how Docker works (e.g. how to make an image, how to make a container and start it)

Meningitis Risk Assessment

A clinical decision-support web application that stratifies meningitis risk into low, moderate, or high tiers based on CSF and blood markers. Powered by a Cost-Sensitive Random Forest model, the tool provides per-class probabilities and confidence scores for evaluated cases. It supports both single-patient evaluations and batch CSV uploads featuring automated validation, data visualization, and downloadable results. In addition to quantitative risk scoring, the platform integrates Google AI Studio's Gemini API alongside a built-in clinical knowledge base fallback to supply contextual interpretations and actionable next steps.

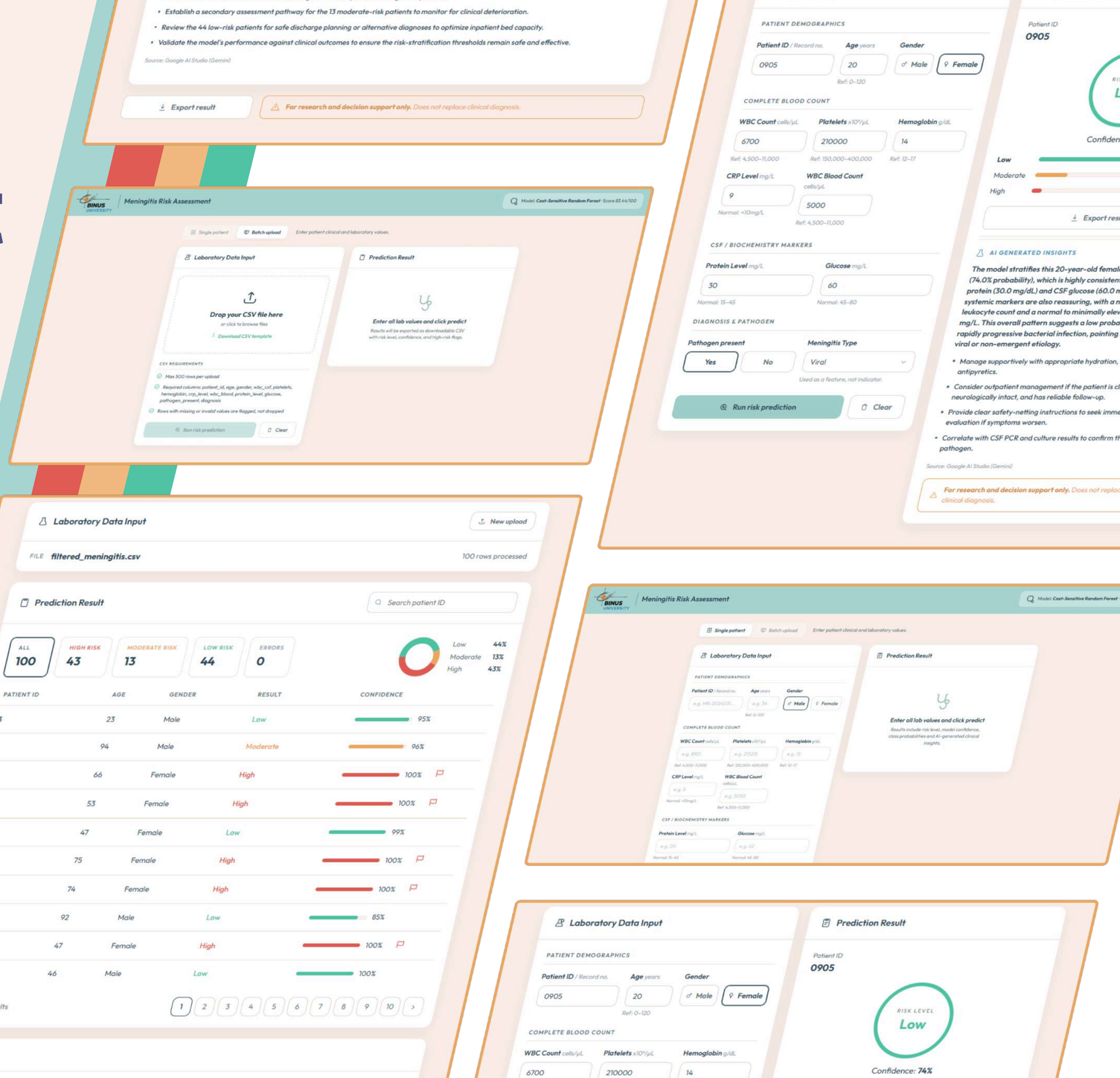
tech stack

- HTML, CSS, JS
- Google AI Studio API
- Flask

links

chrysan95

chrysan.pythonanywhere.com



goals

- Build a meningitis risk assessment web app that turns a validated Jupyter notebook model into a deployable tool
- Build an app that supports both single-patient input and batch CSV upload for clinical trial use
- Integrate Google AI Studio (Gemini) to generate clinical insights, with a rule-based fallback when the API is unavailable
- Design a clean, easy-to-read interface so medical professionals can interpret results at a glance

difficulties

- Reproducing the notebook's preprocessing exactly (one-hot encoding, SMOTE, cost-matrix decision rule) so the web app's predictions matched the validated model.
  Fix: Moved training into `train_model.py` and saved the model, feature order and cost matrix together in one bundle
- Sending every patient's data to Gemini for the batch summary would be slow and token-heavy.
  Fix: Only aggregate risk counts are sent, never individual patient rows

what i learned

- I learned how to turn a machine learning notebook into a working Flask web app with API endpoints
- I learned how to use the Google AI Studio API, prompt it for structured JSON output, and fall back gracefully when it fails
- I learned to keep validation rules in one place (`app.py`) so the frontend and backend always stay in sync
- I learned to design AI features with token efficiency and patient privacy in mind
- I learned how to build interactive UI with vanilla JS and CSS (drag-and-drop upload, filtering, pagination, a donut chart)